

MCL-BB

Building Violation Detection, Challan,
Notice & Enforcement
Workflow Management System

PRODUCT & IMPLEMENTATION PLAN

Version 3 · September 14, 2026
Ludhiana Municipal Corporation

STATUS

Active Development

01 Product Direction

What MCL-BB is and who it serves

MCL-BB is a **Building Violation Detection, Challan, Notice and Enforcement Workflow Management System** for Ludhiana Municipal Corporation. It handles external complaint/document intake, BI field enforcement, and includes a dedicated **BI Performance Analytics** layer to drive accountability and field productivity.

Role Model

Role	Responsibilities
Operator	External case intake, document upload, OCR review
BI	Field visits, complaint updates (minimal), challans, reinspections
ATP	Zone dashboard, complaint approval/close, challan monitoring, escalation
MTP	Higher-level review, escalation participation
JC	City-level monitoring, analytics, final authority

Architecture Decision

One backend, one frontend codebase. No split. Backend owns: RBAC, workflow transitions, challan rules, notice scheduling, notification recipients, audit history, escalation, analytics queries. Frontend owns: role-based UI, dashboards, mobile layouts, forms, PWA installation, push subscription, notification display.

Current Stack

Layer	Technology
Frontend	React 19 + TypeScript + Vite
Backend	Node.js + Express 5 + TypeScript
Database	PostgreSQL via pg
OCR	Mistral OCR
AI Extraction	Anthropic Claude structured extraction
File Storage	Google Drive API (driveService.ts)
Sync	Google Sheets sync
PWA	Web Push, service worker, manifest, basic caching

02 Two Case Entry Paths

How cases enter the system — complaint-driven vs field-discovered

Path A — External Case (Complaint-Driven)

Operator / ATP / authorized user receives complaint via mail, document, news, or direct intake → creates case → location mapping → BI assignment.

Existing OCR pipeline retained: upload → Mistral OCR → Claude structured extraction → operator review/edit → registration.

BI's role in complaint resolution is intentionally minimal:

- BI visits the site
- Uploads one **geotagged photo** (evidence of visit)
- Submits a **short description / resolution report**
- ATP reviews and decides: Close or Pending further action

Note: BI does NOT fill a full inspection form for complaint resolution.

That data is captured in the BI Field Visit (Path B)
if enforcement is warranted.

Path B — BI Field Visit (Proactive Enforcement)

BI independently discovers a suspected violation in the field → opens Field Visit form → fills complete inspection data → submits to BI ledger.

This is the **primary source** of enforcement data and BI analytics.

Attribute	Path A — External Case	Path B — BI Field Visit
Trigger	Complaint / document intake	BI field discovery
Form complexity	Minimal (photo + description)	Full inspection data
GPS required	Yes (geotagged photo)	Yes + accuracy field
Challan path	Via ATP decision → Path B	Direct — no ATP pre-approval
Analytics source	No	Yes — primary source
complaint_id FK	Required	Nullable (proactive)

Both paths enter the same operational lifecycle after their entry point. A field visit may optionally reference a complaint (complaint_id nullable FK) if the visit was triggered by one — but the data structures remain separate because the data shapes are fundamentally different.

03 Database Schema

Core tables — never merged across the two paths

Two separate tables for the two BI roles. The schema is designed so Path A and Path B data never mix — they share officers as FK but nothing else.

complaints

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
complaint_id VARCHAR UNIQUE -- e.g. CMP-2026-001
registration_source VARCHAR -- manual | document | field
name / phone / email VARCHAR
zone / block / ward VARCHAR
address TEXT
title / description VARCHAR / TEXT
drive_folder_url TEXT -- never exposed to frontend
status VARCHAR DEFAULT Registered
bi_officer_id UUID REFERENCES officers(id)
atp_officer_id UUID REFERENCES officers(id)
created_at / updated_at TIMESTAMPTZ
```

officers

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
name VARCHAR NOT NULL
phone / email VARCHAR
role VARCHAR NOT NULL -- BI | ATP | MTP | JC | Operator
zone VARCHAR
block VARCHAR[] -- array: BIs cover multiple blocks
employee_id VARCHAR UNIQUE
created_at TIMESTAMPTZ
```

bi_complaint_updates (Path A)

BI's update on a complaint: photo + description only.

```
id UUID PRIMARY KEY
complaint_id UUID NOT NULL REFERENCES complaints(id)
bi_officer_id UUID NOT NULL REFERENCES officers(id)
geotagged_photo TEXT -- Drive file URL (proxied, never direct)
gps_lat NUMERIC(10,7)
gps_lng NUMERIC(10,7)
description TEXT NOT NULL
submitted_at TIMESTAMPTZ DEFAULT now()
```

bi_field_visits (Path B)

Full proactive enforcement inspection record.

```
id UUID PRIMARY KEY
complaint_id UUID REFERENCES complaints(id) -- NULLABLE
bi_officer_id UUID NOT NULL REFERENCES officers(id)
atp_officer_id UUID REFERENCES officers(id)
zone / block / ward VARCHAR
gps_lat / gps_lng NUMERIC(10,7)
gps_accuracy NUMERIC(8,2)
building_type VARCHAR
property_address TEXT
violator_name / violator_contact VARCHAR
violation_description TEXT
pmc_section VARCHAR -- e.g. 270(1) | 269
form_type VARCHAR -- Form1 | Form2
notice_issued_at TIMESTAMPTZ
notice_period_days INTEGER -- NEVER hardcoded
reminder_at TIMESTAMPTZ -- issued_at + (period/2)
expires_at TIMESTAMPTZ -- issued_at + period
notice_status VARCHAR -- Active | Resolved | Expired
drive_folder_url TEXT -- subfolder for this visit
submitted_at TIMESTAMPTZ
```

visit_evidence & challans

```
-- visit_evidence
id UUID PRIMARY KEY
visit_id UUID NOT NULL REFERENCES bi_field_visits(id)
file_type VARCHAR -- inspection_photo | notice_photo | challan_image
drive_url TEXT -- internal only, never sent to frontend
uploaded_at TIMESTAMPTZ

-- challans
id UUID PRIMARY KEY
visit_id UUID NOT NULL REFERENCES bi_field_visits(id)
complaint_id UUID REFERENCES complaints(id) -- nullable
bi_officer_id UUID NOT NULL REFERENCES officers(id)
challan_number VARCHAR UNIQUE
challan_image TEXT -- Drive URL
issued_at TIMESTAMPTZ DEFAULT now()
status VARCHAR DEFAULT Issued -- Issued | Resolved | Escalated
```

complaint_status_log (Audit Trail)

```
id UUID PRIMARY KEY
complaint_id UUID NOT NULL REFERENCES complaints(id)
old_status VARCHAR
new_status VARCHAR NOT NULL
changed_by UUID REFERENCES officers(id)
note TEXT
changed_at TIMESTAMPTZ DEFAULT now()
```

04 Workflow & State Machine

Status lifecycle, valid transitions, and workflow events

Status Lifecycle

Status	Meaning
REGISTERED	Case created, not yet assigned
ASSIGNED	BI officer mapped to complaint
UNDER_INSPECTION	BI actively working the case
COMPLAINT_UPDATE_SUBMITTED	BI submitted photo + description
PENDING_ENFORCEMENT	ATP decided: enforcement needed
FIELD_VISIT_RECORDED	BI field visit submitted to ledger
CHALLAN_ISSUED	Challan created and Drive upload done
NOTICE_ACTIVE	Notice period running
INVALID_NO_ACTION	Visit recorded, no violation found
RESOLVED	Corrective action completed before expiry
NOTICE_EXPIRED	Notice period elapsed with no action
REINSPECTION_REQUIRED	BI must revisit
REINSPECTION_COMPLETED	Second visit submitted
ESCALATED	Sent to ATP/MTP/JC for action
FINAL_ACTION_RECORDED	Government enforcement outcome logged
CLOSED	Terminal state

Valid Transitions (backend-enforced)

```
REGISTERED → ASSIGNED
ASSIGNED → UNDER_INSPECTION
UNDER_INSPECTION → COMPLAINT_UPDATE_SUBMITTED
COMPLAINT_UPDATE_SUBMITTED → CLOSED | PENDING_ENFORCEMENT
PENDING_ENFORCEMENT → FIELD_VISIT_RECORDED
FIELD_VISIT_RECORDED → CHALLAN_ISSUED | INVALID_NO_ACTION
INVALID_NO_ACTION → CLOSED
CHALLAN_ISSUED → NOTICE_ACTIVE
NOTICE_ACTIVE → RESOLVED | NOTICE_EXPIRED
RESOLVED → CLOSED
NOTICE_EXPIRED → REINSPECTION_REQUIRED
REINSPECTION_REQUIRED → REINSPECTION_COMPLETED
REINSPECTION_COMPLETED → ESCALATED | CLOSED
ESCALATED → FINAL_ACTION_RECORDED
FINAL_ACTION_RECORDED → CLOSED
```

Workflow Events

Event	Event
CASE_CREATED	CASE_ASSIGNED
BI_COMPLAINT_UPDATE_SUBMITTED	BI_FIELD_VISIT_CREATED
BI_FIELD_VIOLATION_RECORDED	CHALLAN_ISSUED
NOTICE_STARTED	NOTICE_REMINDER_DUE
VIOLATION_RESOLVED	NOTICE_EXPIRED
REINSPECTION_REQUIRED	REINSPECTION_COMPLETED
FOLLOWUP_REPORT_SUBMITTED	CASE_ESCALATED
FINAL_ACTION_RECORDED	CASE_CLOSED

05 BI Analytics & Notifications

Metrics, dashboards, and event-driven notification rules

BI Performance Analytics

Analytics are per-BI, computed from real records. Purpose: accountability and productivity visibility for ATP/MTP/JC dashboards. **No separate analytics write path — computed from `bi_field_visits` and `challans`.**

Metric	Source Table / Filter
Total field visits	<code>bi_field_visits</code>
Challans issued	<code>challans</code>
Proactive cases (no complaint)	<code>bi_field_visits WHERE complaint_id IS NULL</code>
Complaint updates submitted	<code>bi_complaint_updates</code>
Notices resolved before expiry	<code>bi_field_visits WHERE notice_status = Resolved</code>
Notices expired (no action)	<code>bi_field_visits WHERE notice_status = Expired</code>
Reinspections done	<code>bi_field_visits linked to reinspection events</code>
Avg. assignment-to-update time	<code>complaints + bi_complaint_updates</code>

Dashboard Views

Dashboard	Audience	Content
BI's own dashboard	BI officer	Visits, challans, pending notices, today's tasks
ATP dashboard	ATP	All BIs in zone ranked by activity, complaint queue
MTP / JC dashboard	MTP, JC	City-wide BI performance, zone comparisons, escalation

Notifications

After complaint update submitted (Path A)

- ATP of the zone — notified immediately

After challan issuance (Path B)

- Person challan was issued against
- Property owner (dedup by phone/name — skip if same as above)
- Zone ATP
- BI gets confirmation

Midway reminder

- Challan recipient

After notice expiry

- BI (reinspection task)
- ATP + configurable escalation recipients (MTP, JC)

Notice Period Configuration

Notice timing is **configurable** — **never hardcoded**. Expected range: 3–7 days, set by admin.

Field	Value / Rule
<code>issued_at</code>	Timestamp of notice issuance
<code>notice_period_days</code>	Admin-configured integer (3–7 days typical)
<code>reminder_at</code>	<code>issued_at</code> + (<code>notice_period_days</code> / 2) days
<code>expires_at</code>	<code>issued_at</code> + <code>notice_period_days</code> days
<code>notice_status</code>	Active Resolved Expired

06 Implementation Order & Non-Negotiable Rules

Build sequence and hard constraints

Implementation Order

#	Task	Status
1	Preserve current complaint registration	Done
2	Add bi_complaint_updates table + POST /api/complaints/:id/bi-update	Next
3	Add bi_field_visits + visit_evidence + challans tables	
4	Add POST /api/inspections (field visit submission)	
5	Add complaint status state machine (PATCH /api/complaints/:id/status)	
6	Add complaint_status_log audit trail	
7	Add authentication / RBAC foundation	
8	Add role-aware dashboards + workflow history views	
9	Add challan creation + challan image upload to Drive	
10	Add push subscriptions + event-driven notifications	
11	Add configurable notice scheduler + midpoint reminder	
12	Add reinspection + escalation workflow	
13	Add BI analytics endpoints + ATP/JC analytics dashboards	
14	Add MTP/JC escalation views	

Non-Negotiable Rules

- BI can issue challan **without ATP pre-approval**
- BI complaint update (Path A) and BI field visit (Path B) are **separate tables, separate routes, separate forms — never merged**
- bi_field_visits.complaint_id is nullable — proactive visits have no complaint parent
- Drive URLs are **never exposed directly to the frontend** — backend proxy only
- Critical authority rules are backend-enforced, not frontend-gated only
- Notifications are event-driven
- Notice timing is configurable — never hardcoded
- Analytics are computed from real records — no separate analytics write path
- ATP and higher authorities share the same application
- BI gets mobile-first PWA experience from the same frontend codebase

BI Authority Rules

BI can issue a challan without ATP pre-approval.

BI complaint update (Path A) triggers ATP notification automatically.

BI field visit (Path B) goes to BI ledger — ATP notified only after challan issuance.

Analytics data is read-only for BI — they cannot edit their own records.

07 Target Code Structure

Frontend and backend directory layout

Frontend (React 19 + TypeScript + Vite)

```
Frontend/src/
  ■■■ app/
  ■■■ modules/
    ■ ■■■ complaints/
    ■ ■■■ violations/
    ■ ■■■ inspections/ ← bi_field_visits forms + ledger view
    ■ ■■■ challans/
    ■ ■■■ notices/
    ■ ■■■ reports/
    ■ ■■■ notifications/
    ■ ■■■ analytics/ ← BI performance dashboards
  ■■■ dashboards/
  ■ ■■■ bi/ ← mobile-first, visit + challan + today tasks
  ■ ■■■ atp/ ← zone overview, complaint queue, BI rankings
  ■ ■■■ mtp/
  ■ ■■■ jc/ ← city-wide analytics + escalation
  ■■■ auth/
  ■■■ layout/
  ■■■ shared/
  ■■■ pwa/
  ■■■ App.tsx
```

Backend (Node.js + Express 5 + TypeScript)

```
server/
  ■■■ app.ts
  ■■■ routes/
  ■■■ modules/
    ■ ■■■ complaints/
    ■ ■■■ violations/
    ■ ■■■ inspections/ ← bi_field_visits + visit_evidence
    ■ ■■■ challans/
    ■ ■■■ notices/
    ■ ■■■ reports/
    ■ ■■■ notifications/
    ■ ■■■ users/
    ■■■ workflow/
    ■ ■■■ events/
    ■ ■■■ handlers/
    ■ ■■■ state-machine/ ← STATUS_TRANSITIONS map lives here
  ■■■ services/
    ■ ■■■ analyticsService.ts ← BI performance query functions
    ■ ■■■ driveService.ts
    ■ ■■■ ocrService.ts
```

```
■ ■■■ claudeService.ts
■ ■■■ locationMapping.ts
■ ■■■ officerMapping.ts
■ ■■■ googleSheetsService.ts
■■■ db/
■■■ uploads/
■■■ shared/
```

Migrate incrementally — no big-bang refactor.